Introduction to Queue and Linked List

Course Name: Data Structures
Course Code: E1PY203B
Instructor Name: Dr. Kanchan Naithani

Unit-02

Queue: Representation and Implementation of Queue (Array and Linked list) - Create, Add, Delete, Full and Empty Circular Queue – Dequeue – Priority Queue.

Lists: Representation and Implementation of Singly Linked Lists - Doubly Linked Lists – Circular Linked Lists, Operations on a Linked List. Insertion, Deletion, Traversal Application of Lists: Polynomial Representation and Addition.

Concepts of Queue

A Queue Data Structure is a fundamental concept in computer science used for storing and managing data in a specific order. It follows the principle of “First in, First out” (FIFO), where the first element added to the queue is the first one to be removed. Queues are commonly used in various algorithms and applications for their simplicity and efficiency in managing data flow.

It operates like a line where elements are added at one end (**rear**) and removed from the other end (**front**).

Basic Operations of Queue Data Structure

- **Enqueue (Insert)**: Adds an element to the rear of the queue.
- **Dequeue (Delete)**: Removes and returns the element from the front of the queue.
- **Peek**: Returns the element at the front of the queue without removing it.
- **Empty**: Checks if the queue is empty.
- **Full**: Checks if the queue is full.

Basic Operations of Queue Data Structure

```
ENQUEUE(queue, front, rear, value):
   if (rear == SIZE - 1):
      print "Overflow"
      return

   if (front == -1):
      front = 0

   rear = rear + 1
   queue[rear] =value
   print "Inserted", value
```

(Base Condition for Queue)

```
DEQUEUE(queue, front, rear):
   if (front == -1 or front > rear):
      print "Underflow"
      return

   value = queue[front]
   front = front + 1

   if (front > rear):
      front ← -1
      rear ← -1

   print "Deleted", value
```

Applications of Queue

- **Task scheduling** in operating systems
- **Data transfer** in network communication
- **Simulation** of real-world systems (e.g., waiting lines)
- **Priority queues** for event processing queues for event processing

Priority Queue

A priority queue is a queue where each element has a priority associated with it. Elements with higher priority are dequeued before elements with lower priority, regardless of the order in which they were enqueued.

Note: It does **not** follow the strict **FIFO (First In, First Out)** rule like a regular queue.

Operations:

- **Insertion (Enqueue):** Insert an element with its priority.
- **Deletion (Dequeue):** Remove and return the element with the highest priority.
- **Peek:** View the element with the highest priority without removing it.
- **C Implementation:** Priority queues can be implemented using various data structures like heaps, binary search trees, or arrays. One common approach is to use a min-heap or max-heap to maintain the priority order.

Priority Queue

```
INSERT(PQ, value, priority)
1. If size == MAX:
      Print "Overflow! Queue is full"
      Exit
2. Set i ← size - 1
3. While i ≥ 0 and PQ[i].priority < priority:
      PQ[i + 1] = PQ[i]      // Shift lower-priority elements right
      i = i - 1
4. PQ[i + 1].data = value
5. PQ[i + 1].priority = priority
6. size = size + 1
7. Print "Element inserted successfully"
```

(Base Condition for Priority Queue)

```
DELETE(PQ)
1. If size == 0:
      Print "Underflow! Queue is empty"
      Exit
2. value = PQ[0].data
3. For i = 0 to size - 2:
      PQ[i] = PQ[i + 1]      // Shift all elements left
4. size = size - 1
5. Print "Deleted element is", value
```

Priority Queue VS Queue

Feature	**Queue**	**Priority Queue**
Order of Processing	First In, First Out (FIFO)	Based on **priority**, not arrival time
Insertion	Always at the **rear**	Can be **anywhere**, based on priority
Deletion	Always from the **front**	Always removes the **highest priority** element
Use Cases	Scheduling print jobs, managing tasks in line	CPU scheduling, emergency rooms, Dijkstra's algorithm
Priority Support	No	Yes – each element has a priority level

Dequeue (Double-Ended Queue)

A dequeue (pronounced "deck") is a queue-like data structure that supports insertion and deletion at both the front and the rear ends.

Operations:

1. **Insertion at Front and Rear**: Insert an element at either the front or the rear end.
2. **Deletion from Front and Rear**: Remove and return an element from either the front or the rear end.
3. **Peek Front and Rear**: View the element at either the front or the rear end without removing it.

C Implementation: Dequeues can be implemented using doubly linked lists to efficiently support insertion and deletion at both ends.

Dqueue (Double-Ended Queue)

Insertion

Algorithm for Insertion at rear end

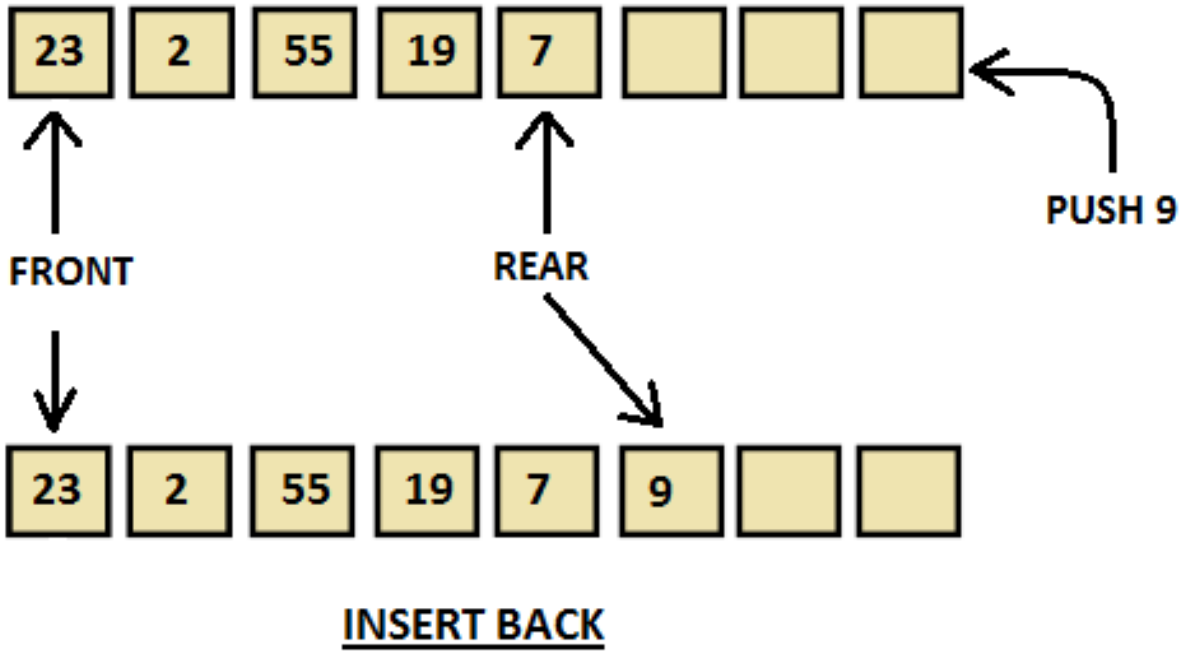
1. If rear == size - 1 then
   Output "Overflow – Cannot insert at rear"
   Exit

2. If front == -1 and rear == -1 then
   front ← 0
   rear ← 0

3. Else
   rear ← rear + 1

4. DQ[rear] ← item

5. End



Dqueue (Double-Ended Queue)

Insertion

Algorithm for Insertion at front end

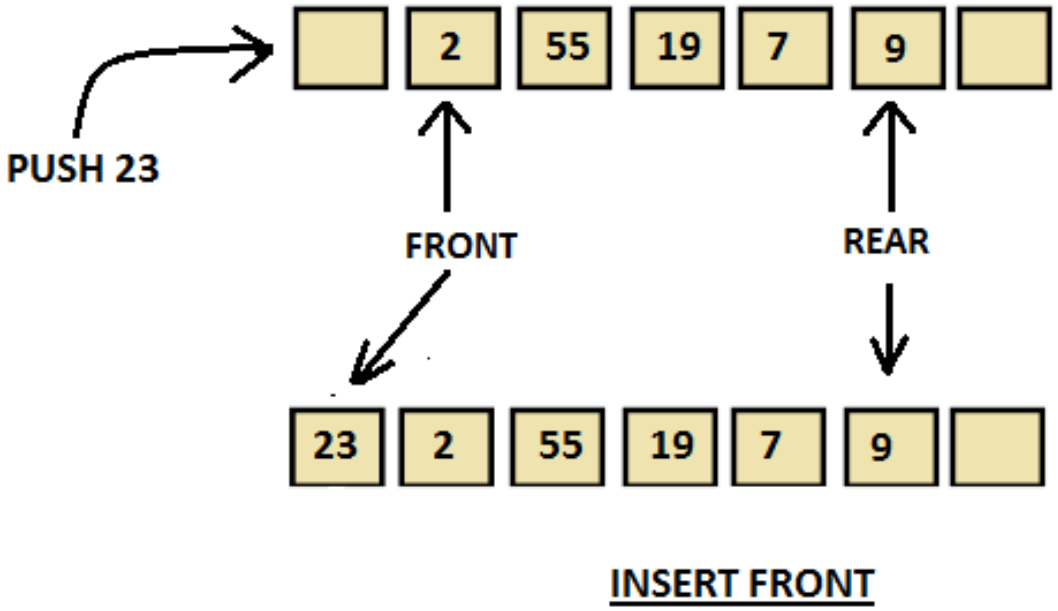
1. If front == 0 then
   Output "Overflow – Cannot insert at front"
   Exit

2. If front == -1 and rear == -1 then
   front ← 0
   rear ← 0

3. Else
   front ← front - 1

4. DQ[front] ← item

5. End



Dqueue (Double-Ended Queue)

Deletion

Algorithm for Deletion from front end

1. If front == -1 then
   Output "Underflow – Deque is empty"
   Exit

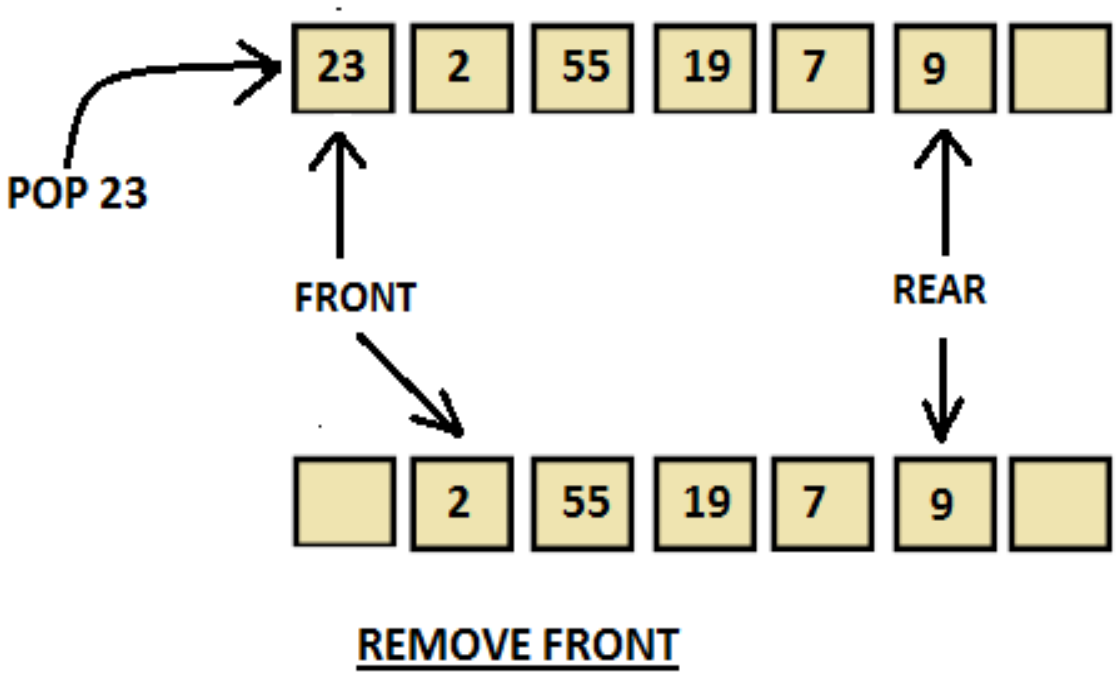
2. Element ← DQ[front] // Store the element to be deleted (optional)

3. If front == rear then
   front ← -1
   rear ← -1 // Deque becomes empty after deletion

4. Else
   front ← front + 1

5. Output "Deleted element is", Element (optional)

6. End



Dqueue (Double-Ended Queue)

Deletion

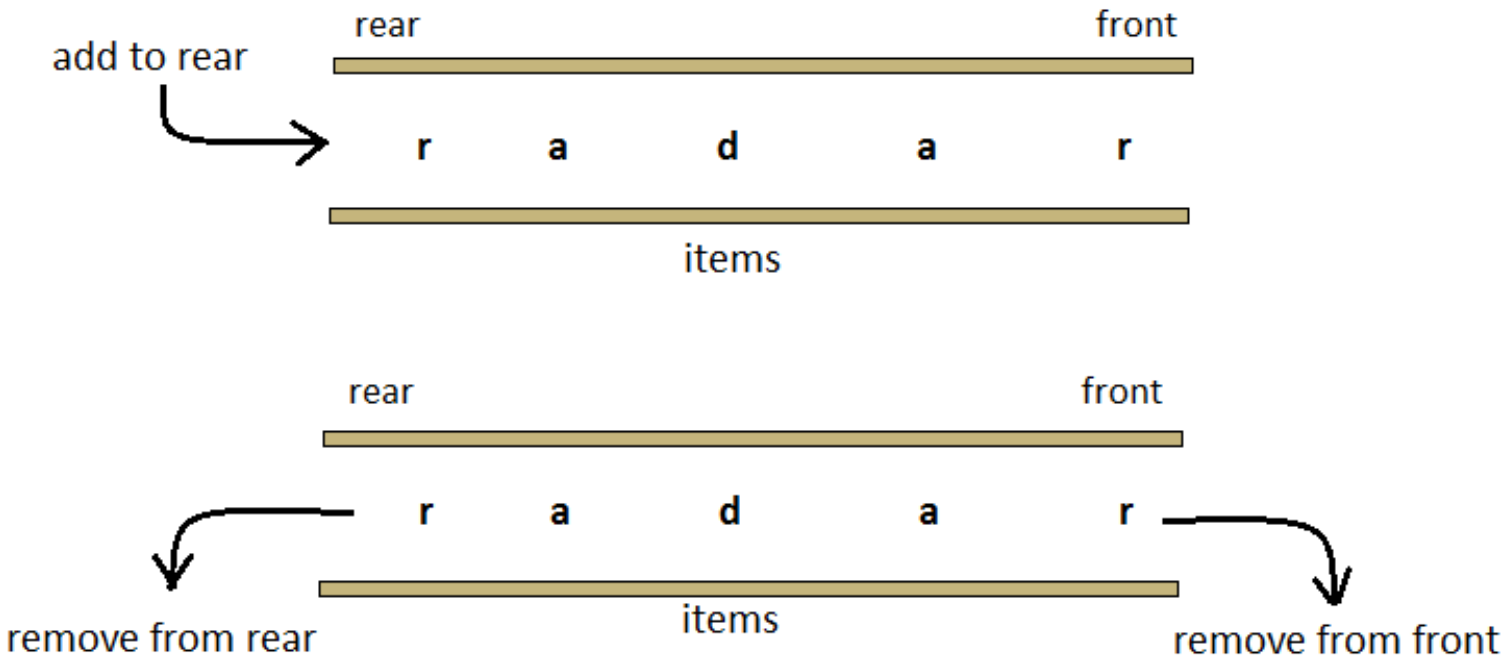
Algorithm for Deletion from rear end

```
1. If rear == -1 then
      Output "Underflow – Deque is empty"
      Exit

2. Element ← DQ[rear]    // Store the element to be deleted (optional)

3. If front == rear then
      front ← -1
      rear ← -1          // Deque becomes empty after deletion

4. Else
      rear ← rear - 1

5. Output "Deleted element is", Element (optional)

6. End
```

Applications of Dqueue (Double-Ended Queue)

Palindrome Checker

Add "radar" to rear



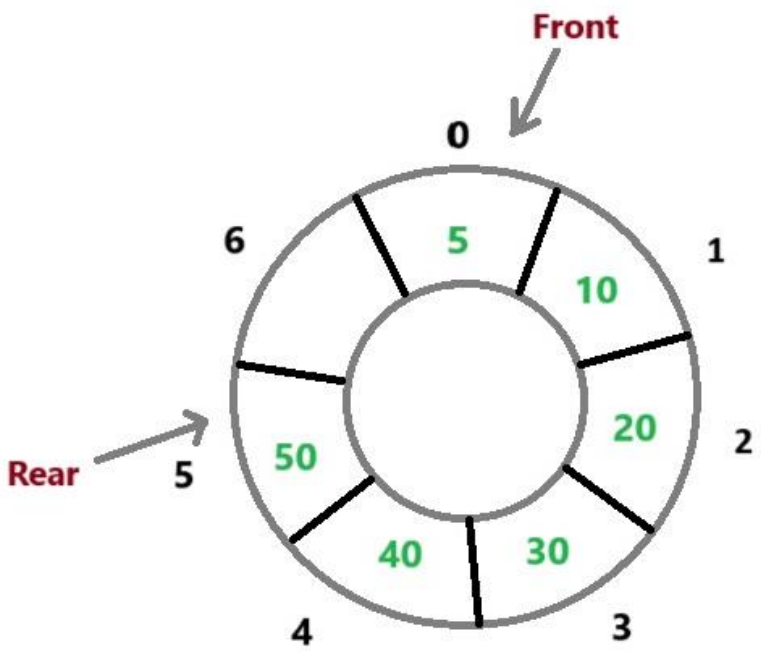
Remove from front & rear

Dqueue Vs Queue

Feature	Queue	Deque (Double-Ended Queue)
Definition	A linear data structure that follows **FIFO** (First In First Out).	A linear data structure where insertion and deletion can occur from **both ends.**
Insertion	Only at the **rear.**	At **front** and **rear** both.
Deletion	Only from the **front.**	From **front** and **rear** both.
Types	Simple Queue, Circular Queue, Priority Queue	Input-restricted and Output-restricted deque.
Flexibility	Less flexible (only one end for insertion, one for deletion).	More flexible (both ends can be used).
Use Cases	Task scheduling, printer queue, call center queue.	Undo operations, palindrome checking, sliding window problems.
Implementation	Easier to implement than deque.	Slightly more complex due to two-end operations.

Circular Queue

A circular queue is a queue-like data structure where the last element is connected back to the first element, forming a circular arrangement. This allows efficient use of space and avoids the need to shift elements when dequeuing.



Operations:

1. **Enqueue:** Add an element to the rear end of the queue.
2. **Dequeue:** Remove and return the element from the front end of the queue.
3. **Overflow and Underflow Handling:** Handle situations when the queue becomes full (overflow) or empty (underflow).

C Implementation: Circular queues can be implemented using arrays or linked lists. When using arrays, modular arithmetic is often used to wrap around indices when reaching the end of the array.

Circular Queue

Insertion

```
Procedure Enqueue(item)
   if ( (rear + 1) % SIZE == front ) then
      print "Queue is Full (Overflow)"
      return
   else if (front == -1 and rear == -1) then
            front ← 0              // Queue is empty, first insertion
            rear ← 0
   else
            rear ← (rear + 1) % SIZE                    // Move rear to the next circular position
end if
   queue[rear] ← item
   print "Item inserted:", item
End Procedure
```

(Base Condition for Circular Queue)

Circular Queue

Deletion

```
Procedure Dequeue()
   if (front == -1 and rear == -1) then
      print "Queue is Empty (Underflow)"
      return
   end if

   item ← queue[front]

   if (front == rear) then
      front ← -1                // Only one element was in the queue
      rear ← -1
   else
       front ← (front + 1) % SIZE          // Move front to the next circular position
   end if

   print "Item deleted:", item
   return item
End Procedure
```

(Base Condition for Circular Queue)

Circular Queue Vs Queue

Feature	Linear Queue	Circular Queue
Structure	Linear (front to rear in a straight line)	Circular (rear wraps around to the front)
Space Utilization	Inefficient – can’t reuse space after deletion	Efficient – reuses freed space by wrapping around
Overflow Condition	When rear == MAX - 1	When (rear + 1) % MAX == front
Insertion	Only at the rear end	At rear, but wraps to front if there's space
Deletion	Only at the front end	At front, with wrap-around logic
Memory Usage	May waste memory (if elements are deleted)	Optimized use of memory
Implementation	Easier	Slightly complex due to circular logic
Use Cases	Basic queues, print queues, ticket systems	CPU scheduling, buffer management, embedded systems
Front/Rear Resetting	Needs manual resetting when all elements are deleted	Automatically wraps and reuses the array

Summary of Types of Queue

Queue Type	Initial Front Value	Initial Rear Value	Front Movement	Rear Movement	Special Characteristics	Example Usage
Simple Queue	Front = 0	Rear = -1	Increments on each deletion (Front++)	Increments on each insertion (Rear++)	Fixed size, no reuse of freed space	Printer queue, call center systems
Circular Queue	Front = 0	Rear = -1	(Front + 1) % Size on deletion	(Rear + 1) % Size on insertion	Space-efficient by reusing freed slots	CPU scheduling
Priority Queue	Front = 0 (logical)	Rear = -1 (logical)	Based on priority, not position-based	Inserted based on priority rules	Elements dequeued by priority rather than position	Task scheduling, Dijkstra's algorithm
Deque (Double-Ended Queue)	Front = -1	Rear = 0	Front++ or Rear-- depending on operation	Front-- or Rear++ depending on operation	Insert/delete at both ends; flexible	Palindrome checker, undo operations

Summary of Types of Queue

Queue Type	Base Condition	Meaning / Usage
Simple Queue	front == NULL	Queue is **empty**
	rear == NULL	Queue is **empty**
Dequeue	After dequeue(): if (front == NULL) rear = NULL;	Queue became empty after deletion
	front == NULL && rear == NULL	Dequeue is **empty**
	front == rear	Only **one element** in the dequeue
	insertFront() or insertRear() with full condition	(In static dequeue) **overflow** if queue is full
Priority Queue	front == NULL	Priority Queue is **empty**
	new_node->priority > current->priority	Insert before current node (based on higher priority)
	traverse till correct priority position	To maintain sorted order by priority
Circular Queue	front == -1 && rear == -1	Queue is **empty** (initial condition)
	(rear + 1) % size == front	Queue is **full**
	front == rear (after dequeue)	Queue has **one element** before becoming empty
	After last deletion: front = -1; rear = -1;	Reset to **empty state**

Introduction to Linked Lists

The Linked Lists are linear data structures where the data is not stored at contiguous memory locations so we can only access the elements of the linked list in a sequential manner. Linked Lists are used to overcome the shortcoming of arrays in operations such as deletion, insertion, etc.

A linked list is a linear data structure that includes a series of connected nodes. Here, each node stores the **data** and the **address** of the next node. For example,

A linked list starts with a **head** node which points to the first node. Every node consists of data which holds the actual data (value) associated with the node and a next pointer which holds the memory address of the next node in the linked list. The last node is called the tail node in the list which points to **null** indicating the end of the list.

Components of Linked Lists

Node Structure: A node in a linked list typically consists of two components:

- **Data:** It holds the actual value or data associated with the node.
- **Next Pointer:** It stores the memory address (reference) of the next node in the sequence.

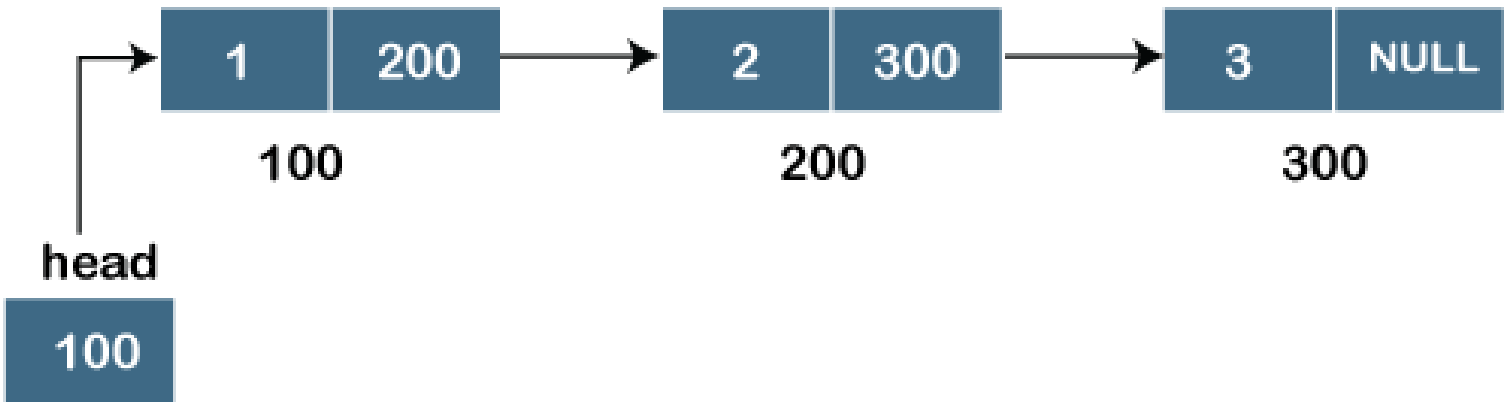
Head and Tail: The linked list is accessed through the head node, which points to the first node in the list. The last node in the list points to NULL or nullptr, indicating the end of the list. This node is known as the tail node.

Linked Lists vs Arrays

- In case of arrays, the size is given at the time of creation and so arrays are of fixed length
- Linked lists are dynamic in size and any number of nodes can be added in the linked lists dynamically.
- An array can accommodate similar types of data types
- Linked lists can store various nodes of different data types.

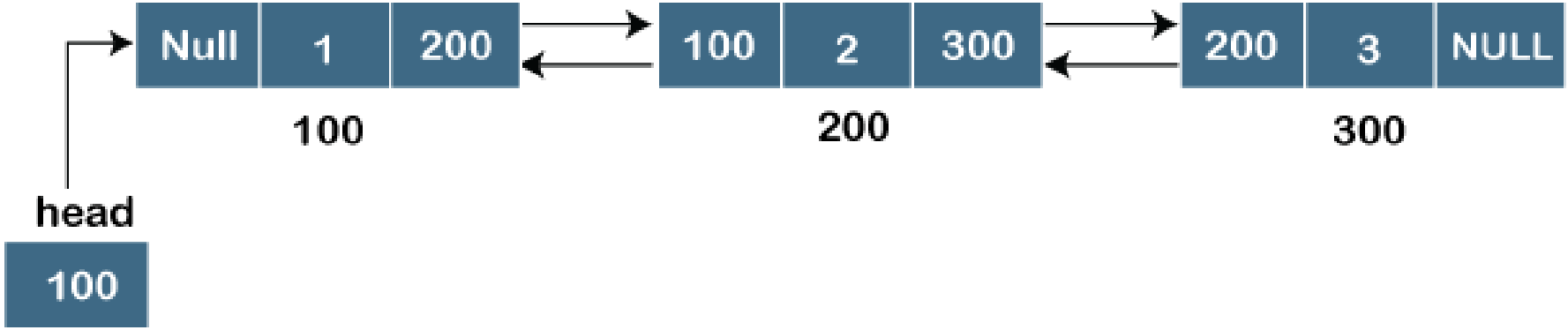
Types of Linked List

- **Singly Linked Lists -** The singly linked list is a data structure that contains two parts, i.e., one is the data part, and the other one is the address part, which contains the address of the next or the successor node. The address part in a node is also known as a pointer. Suppose we have three nodes, and the addresses of these three nodes are 100, 200 and 300 respectively. The representation of three nodes as a linked list is shown in the below figure:

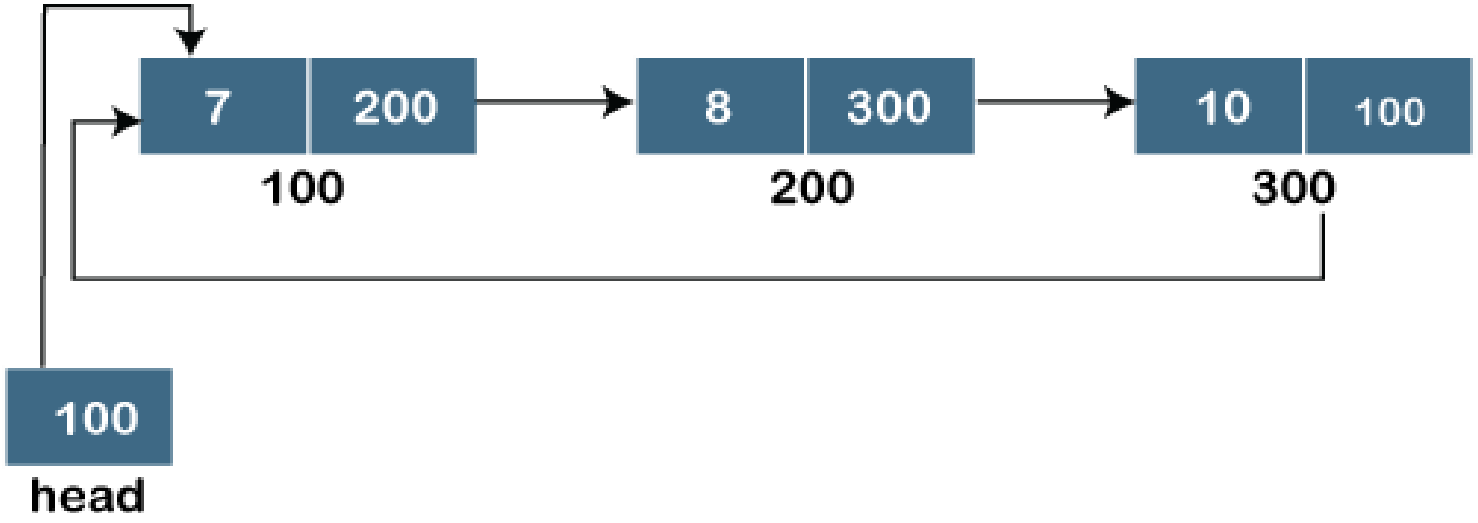


Types of Linked List

- **Doubly Linked Lists -** a doubly linked list is a list that has three parts in a single node, includes one data part, a pointer to its previous node, and a pointer to the next node. Suppose we have three nodes, and the address of these nodes are 100, 200 and 300, respectively. The representation of these nodes in a doubly-linked list is shown below:



- **Circular Linked Lists -** the circular linked list is a list in which the last node connects to the first node, so the link part of the last node holds the first node's address. The circular linked list has no starting and ending node. We can traverse in any direction, i.e., either backward or forward. The diagrammatic representation of the circular linked list is shown below:



Basic Operations in Linked List

- **Creation –** Creates a node with data and info
- **Insertion** − Adds an element at the beginning of the list.
- **Deletion** − Deletes an element at the beginning of the list.
- **Display** − Displays the complete list.
- **Search** − Searches an element using the given key.
- **Delete** − Deletes an element using the given key.

Creation of Linked List

Each node consists:

1. A data item
2. An address of another node

We wrap both the data item and the next node reference in a struct as:

```
struct node
{
    int data;
    struct node *next;
};
```

Each struct node has a data item and a pointer to another struct node.

Insertion in Linked List

```
void insertAtBeginning (struct Node** head_ref, int new_data)
{
        struct Node* new_ node = (struct Node*)malloc(sizeof(struct Node));        // Allocate memory for new node
        new_node->data = new_data;                                  // Assign data to the new node
        new_node->next = *head_ref;                                 // Set the next of the new node to point to the current head
        *head_ref = new_node;                                       // Move the head to point to the new node
}
```

Insertion in Linked List

```
void insertAtEnd(struct Node** head_ref, int new_data)
{
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));        // Allocate memory
    new_node->data = new_data;                                                // Assign data to the new node
    new_node->next = NULL;                                                    // Set next of new node to NULL
    if (*head_ref == NULL)                                                    // If the list is empty
    {       *head_ref = new_node;                                             // New node becomes the head
            return;     }
    struct Node* last = *head_ref;                                            // Temp pointer to traverse the list
    while (last->next != NULL)                                                // Traverse to the end
        last = last->next;
    last->next = new_node;                                                    // Link last node to new node
}
```

Insertion in Linked List

```
void insertAtPosition(struct Node** head_ref, int new_data, int position)
{
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node)); // Allocate memory
    new_node->data = new_data;        // Assign data
    new_node->next = NULL;

    if (position == 1)                // If inserting at the head
    {
        new_node->next = *head_ref;
        *head_ref = new_node;
        return;
    }

    struct Node* temp = *head_ref;   // Temp pointer to traverse
    for (int i = 1; i < position - 1 && temp != NULL; i++) // Traverse to (position - 1)
        temp = temp->next;

    if (temp == NULL)                // If position is invalid (list too short)
    {
        printf("Position out of range\n");
        return;
    }

    new_node->next = temp->next;     // Link new node to next node
    temp->next = new_node;           // Link previous node to new node
}
```

Printing of Linked List

```
// Function to print the linked list
void printList(struct Node* node) {
    while (node != NULL) {
        printf("%d ", node->data);
        node = node->next;
    }
    printf("\n");
}
```

Searching of Linked List

```
// Function to search for an element in the linked list

struct Node* searchInList(struct Node* head, int key) {
    struct Node* current = head;
    while (current != NULL) {
        if (current->data == key) {
            return current; // Found the element
        }
        current = current->next;
    }
    return NULL; // Element not found
}
```

```
// Function to search for an element in the linked list and return its position

int searchInList(struct Node* head, int key) {
    struct Node* current = head;
    int position = 1;
    while (current != NULL) {
        if (current->data == key) {
            return position; // Found the element
        }
        current = current->next;
        position++;
    }
    return -1; // Element not found
}
```

Deletion of Linked List

```
void deleteNode(struct Node** head, int key) {    // Store head node
    struct Node* temp = *head, *prev;

    // If head node itself holds the key to be deleted

    if (temp != NULL && temp->data == key) {
        *head = temp->next; // Change head
        free(temp);          // Free old head
        return;
    }

    // Search for the key to be deleted, keep track of the previous node

    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }

    // If key was not present in linked list

    if (temp == NULL)
        return;

    // Unlink the node from linked list

    prev->next = temp->next;

    // Free the memory allocated for the node

    free(temp);
}
```

Traversing a Linked List

Displaying the contents of a linked list is done such that we keep moving the temp node to the next one and display its contents.

When temp is NULL, we know that we have reached the end of the linked list so we get out of the while loop.

```
struct node *temp = head;
printf("\n\nList elements are - \n");
while(temp != NULL) {
  printf("%d --->",temp->data);
  temp = temp->next;
}
```

Concept of Overflow and Underflow In Linked List

In a linked list, overflow usually refers to a scenario where there is insufficient memory available to create a new node when attempting to insert an element into the list. This situation can occur if the system's memory is exhausted or if there are memory constraints imposed by the operating system or programming environment. When an overflow condition happens in a linked list, it typically results in a memory allocation failure, and the operation attempting to insert a new node may fail or return an error.

On the other hand, underflow in a linked list generally refers to attempting to perform an operation on an empty list that requires elements to be present. For example, trying to remove a node from an empty list would result in an underflow condition. Underflow can also occur if trying to access or manipulate elements beyond the bounds of the list (e.g., attempting to delete a node at an index greater than the number of nodes in the list).

Note : *The concepts of overflow and underflow are more commonly associated with fixed-size data structures like arrays or stacks, where there are predefined limits on the number of elements that can be stored. In those cases, overflow occurs when attempting to add an element beyond the maximum capacity of the data structure, while underflow happens when attempting to remove an element from an empty data structure. In linked lists, where memory allocation is typically dynamic, the terms are less frequently used in practice.*

Doubly Linked List

A doubly linked list (DLL) is a special type of linked list in which each node contains a pointer to the previous node as well as the next node of the linked list.

```
// Node of a doubly linked list

struct Node {
   int data;

   // Pointer to next node in DLL
   struct Node* next;

   // Pointer to previous node in DLL
   struct Node* prev;
};
```

Advantages of Doubly Linked List

Advantages of Doubly Linked List over the singly linked list:

•A DLL can be traversed in both forward and backward directions.

•The delete operation in DLL is more efficient if a pointer to the node to be deleted is given.

•We can quickly insert a new node before a given node.

•In a singly linked list, to delete a node, a pointer to the previous node is needed. To get this previous node, sometimes the list is traversed. In DLL, we can get the previous node using the previous pointer.

Disadvantages of Linked List

Disadvantages of Doubly Linked List over the singly linked list:

- Every node of DLL Requires extra space for a previous pointer. It is possible to implement DLL with a single pointer though .
- All operations require an extra pointer previous to be maintained. For example, in insertion, we need to modify previous pointers together with the next pointers. For example in the following functions for insertions at different positions, we need 1 or 2 extra steps to set the previous pointer.

Applications of Doubly Linked List

- It is used by web browsers for backward and forward navigation of web pages
- LRU (Least Recently Used) / MRU (Most Recently Used) Cache are constructed using Doubly Linked Lists.
- Used by various applications to maintain undo and redo functionalities.
- In Operating Systems, a doubly linked list is maintained by thread scheduler to keep track of processes that are being executed at that time.

Linked List implementation of queue

Initialization:

1. Define a structure for each node containing the data and a pointer to the next node.
2. Initialize pointers called front and rear to NULL.

Enqueue (Add an element to the queue):

1. Create a new node and allocate memory for it.
2. Set the data of the new node.
3. If the queue is empty (front is NULL), set both front and rear to point to the new node.
4. Otherwise, set the next pointer of the current rear node to the new node and update rear to point to the new node.

Dequeue (Remove an element from the queue):

1. If the queue is empty (front is NULL), print an error message and return.
2. Create a temporary pointer and set it to point to the node at front.
3. Update front to point to the next node.
4. If front becomes NULL after removal, also set rear to NULL.
5. Free the memory of the temporary pointer and return the data from the removed node.

Display:

1. If the queue is empty (front is NULL), print a message.
2. Traverse and print elements from front to rear.

Linked List implementation of Stack

Define a Node structure

•Each node contains:

- •An integer data
- •A pointer next to the next node

Algorithm for Push Operation

Input: Integer value to be pushed

Output: Stack with the new value on top

1.Create a new node

2.Set new_node.data = value

3.Set new_node.next = top

4.Update top = new_node

5.Print "value pushed to stack"

Algorithm for Pop Operation

Input: None

Output: Value popped from the stack

1.If top == NULL:

- •Print "Stack underflow"
- •Exit or return an error value

2.Set temp = top

3.Set top = top.next

4.Store temp.data in popped_value

5.Free the memory of temp

6.Return popped_value

Polynomial Representation Using Linked List

Polynomial representation and addition are classic applications of **linked lists** in data structures. Below is a simple explanation, followed by a complete **menu-driven C program** that demonstrates:

- Representing a polynomial using a linked list
- Performing addition of two polynomials

Each term of the polynomial is stored in a **node** with:

- `coefficient` (int)
- `exponent` (int)
- `next` pointer to the next term

Polynomial: $4x^2 + 3x^2 + 6$

Linked List: [4, 3] -> [3, 2] -> [6, 0]

Step 1: Define the Structure

•Create a structure Node to represent a term in the polynomial.

•Each node should contain:

- •coefficient (integer)
- •exponent (integer)
- •next (pointer to the next node)

Step 2: Initialize the Polynomial

•Initialize the polynomial linked list:

- •head ← NULL

Step 3: Input the Number of Terms

•Read n, the number of terms in the polynomial.

Step 4: Loop to Create Terms

•**For** i ← 1 to n **do**

1. Read the coefficient and exponent of the term.
2. Create a new node newNode using the input values.
3. Set newNode → next ← NULL
4. If head == NULL, set head ← newNode
5. Else:
   - •Traverse the list to the last node.
   - •Link the last node to newNode.

Step 5: Display the Polynomial

•Traverse the linked list from head to NULL.

•For each node:

- •Print coefficient and exponent in the format coeff x^{expo}.

Steps:

Step 1: Start

- Initialize pointers `ptr1 ← poly1`, `ptr2 ← poly2`
- Create a new linked list `result ← NULL` to store the sum.

Step 2: Traverse Both Polynomials

- **While** `ptr1 ≠ NULL` **and** `ptr2 ≠ NULL` **do**
  - **If** `ptr1 → exponent > ptr2 → exponent`:
    - Create a new node with:
      - `coefficient = ptr1 → coefficient`
      - `exponent = ptr1 → exponent`
    - Append the node to `result`
    - Move `ptr1` to the next node
  - **Else If** `ptr1 → exponent < ptr2 → exponent`:
    - Create a new node with:
      - `coefficient = ptr2 → coefficient`
      - `exponent = ptr2 → exponent`
    - Append the node to `result`
    - Move `ptr2` to the next node
  - **Else** (exponents are equal):
    - Add coefficients: `sum = ptr1 → coefficient + ptr2 → coefficient`
    - **If** `sum ≠ 0`, create a new node with:
      - `coefficient = sum`
      - `exponent = ptr1 → exponent`
    - Append the node to `result`
    - Move both `ptr1` and `ptr2` to next nodes

Step 3: Add Remaining Terms

- While `ptr1 ≠ NULL`:
  - Create node with `ptr1 → coefficient` and `ptr1 → exponent`
  - Append to `result`
  - Move `ptr1` to next
- While `ptr2 ≠ NULL`:
  - Create node with `ptr2 → coefficient` and `ptr2 → exponent`
  - Append to `result`
  - Move `ptr2` to next

Step 4: End

- Return or display the `result` polynomial list.

🧠 Example:

Poly1: $4x^3 + 2x^2$

Poly2: $5x^3 + 3x$

Sum: $(4+5)x^3 + 2x^2 + 3x \rightarrow 9x^3 + 2x^2 + 3x$

End of Unit -02